

中山大学计算机学院本科生实验报告

(2024 学年第 1 学期)

课程名称: Data structures and algorithms

任课教师: 张子臻

年级	23 级	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇
电话	13421145433	Email	linhy69@mail2.sysu.edu.cn
开始日期	2024.10.11	完成日期	2024.10.11

第一题

1. 实验题目

方程求解

2. 实验目的

已知函数 $y = e^x + \ln(x) - 1$, 实现函数

```
#include "solve.h"
```

```
long double solve(long double y)
```

```
{
```

```
// here
```

```
}
```

对于传入的 y , 返回 x 值

要求 $f(x)$ 与 y 的误差小于 $1e-6$, 其中 $0 < y < 1e10$

3. 算法设计

1. 直接通过 y 计算 x 比较困难, 则反之, 通过 x 求得 y 去进行比较
2. 利用二分查找法 (数学上为两边夹逼法)
3. 定义一个左值 $left$, 右值 $right$, 不断向中间逼近, 找到一个值可以满足误差小于 $1e-6$ 的结果

4. 程序运行与测试

1. 定义函数 $y=e^x+\ln(x)-1$

```
long double f(long double x, long double y) {  
    return std::exp(x) + std::log(x) - 1.0 - y;  
}
```

2. 设置左值和右值 保证两端足够大

```
long double left = 1e-10; // x > 0  
long double right = 1.0;  
// 通过不断扩大右边界, 确保 f(right) >= 0  
while (f(right, y) < 0) {  
    right *= 2.0;  
    // 为防止右边界无限扩大, 设定一个最大值  
    if (right > 1e10) {  
        // 如果右边界超过了很大的值, 认为无解  
        return -1.0;  
    }  
}
```

3. 二分查找法

4. 控制精度

```
const long double epsilon = 1e-12; // 精度控制  
while (right - left > epsilon)
```

5. 实验总结与心得

1. 第一次接触关于此类数学计算的编程题目, 同数学解题思路不同, 不是用 y 找 x , 而是应该用 x 找 y
2. 找 x 可以通过二分查找法不断逼近, 同时利用函数进行计算得到 y

第二题

1. 实验题目

最大值最小化

2. 实验目的

把一个包含 n 个正整数的序列划分成 m 个连续子序列（每个正整数恰好属于一个序列）。设第 i 个序列的各数之和为 $S(i)$ ，如何让所有 $S(i)$ 的最大值尽量小？

例如序列 1 2 3 2 5 4，划分成 3 个序列的最优方案为 1 2 3 | 2 5 | 4，其中 $S(1)=6$, $S(2)=7$, $S(3)=4$ ，最大值为 7；如果划分成 1 2 | 3 2 | 5 4，则最大值为 9，不如刚才的好。

$n \leq 10^6$ ，所有数之和不超过 10^9 。

3. 算法设计

1. 将输入的子序列划分成 $count$ 个子序列，它们的序列和都不超过 $limit$ 2. 判断 $count$ 是否大于输入的 n 3. 不断更新 $limit$ ，如果 $x0$ 判断为假，则答案比 $x0$ 大，如果 $x0$ 判断为真，则答案比 $x0$ 小。二分查找这个 $limit$ 值，把优化问题转化为判定问题

4. 程序运行与测试

1. 确定二分查找法的上下界，左值 $left$ =最大的元素,右值 $right$ =序列和，保证区间足够大,要找的 $limit$ 就在区间之间

// 确定二分查找的上下界

```
ll left = 0;
ll right = 0;
for(int i = 0; i < n; ++i){
    left = max(left, (ll)s[i]);
    right += s[i];
}
```

//初始化 $left$ 为序列中的最大元素， $right$ 为序列的总和。

2. 在 $left$ 和 $right$ 之间进行二分查找，寻找最小的 $limit$ ，使得可以将序列划分为不超过 m 个子序列。

每次取中间值 mid ，调用 $canDivide$ 判断是否可行。

如果可行，更新 $answer$ 为 mid ，并将 $right$ 调整为 $mid - 1$ ；否则，将 $left$ 调整为 $mid + 1$ 。

```
ll answer = right;
while(left <= right){
    ll mid = left + (right - left) / 2;
    if(canDivide(s, n, m, mid)){
        answer = mid;
        right = mid - 1;
    }
}
```

```

        else{
            left = mid + 1;
        }
    }
}

```

3.定义 canDivide 函数,判断是否可以将序列划分为不超过 m 个子序列,且每个子序列的和不超过 limit

```

bool canDivide(const vector<int> &s, int n, int m, ll limit) {
    int count = 1; // 至少需要一个子序列
    ll current_sum = 0;
    for(int i = 0; i < n; ++i) {
        if(current_sum + s[i] > limit) {
            count++;
            current_sum = s[i];
            if(count > m) return false;
        }
        else {
            current_sum += s[i];
        }
    }
    return true;
}

```

5. 实验总结与心得

1.该实验题难度较大,需要考虑到将优化问题转化为二分查找的判断问题 2.使用二分查找时便要考虑上下界的问题 3.进行判断的函数需要如何设计

第三题

1. 实验题目

Binary Search

2. 实验目的

实现二分查找函数,函数接口如下。

```

#include "binSearch.h"int binSearch(const int s[], const int size, const int target){
    // 请将实现代码添加在这里
}

```

```
}
```

size 为数组 s 的实际大小。

假定 s 非递减有序，如果 s 中存在值为 target 的元素，
则返回最后一次出现的位序号，否则返回-1 表示不存在。

3. 算法设计

1. 设置下标左值 left 和下标右值 right
2. $mid = (left + right) / 2$ 不断二分逼近
3. 找到一个 mid 使得 $s[mid] == target$

4. 程序运行与测试

1. 循环的条件是 $left \leq right$ 而不是 $left < right$

```
while (left <= right)
```

2. 即使找到了一个值 也要使左值 left 继续往后,知道找到最后一次出现的地方

```
loc = mid;  
left = mid + 1; // 继续向右查找，确保找到最后一次出现的位置
```

5. 实验总结与心得

1. 二分查找比较简单，原理搞清了就不难
2. 这道题还需要确保找到的是这个值最后一次出现的位置 所以找到了一个值后还要继续移动左值往后继续查找